Experiment-13

BLOSSOMGifts
ONLINE FLOWER AND GIFT ORDERING SYSTEM

1. Title

BlossomGifts – Online Flower and Gift Ordering System

2. Aim

To design and develop a user-friendly online shopping website for flowers and gifts where customers can browse categorized products, view item details, add products to a shopping cart, schedule delivery dates and times, and place demo orders. The system also outlines administrator functions for managing product inventory, prices, and customer/order information.

3. Software Requirements

Requirement	Specification
Operating system	Windows 10/11, Linux, or macOS
Front-end	HTML5, CSS3, JavaScript
Browser	Google Chrome, Firefox, or Microsoft Edge
Editor	Visual Studio Code / Notepad++
Prototype storage	Browser memory; optional localStorage extension
Production backend	Node.js / PHP / Python with MySQL or equivalent database
Payment and notifications	Payment gateway and email/SMS service integration

4. Problem Description

Customers often need to visit multiple stores or websites to find suitable flowers and gifts. Poor product categorization, unclear product information, and a complicated checkout process can make shopping difficult. Customers also need convenient options to choose delivery dates and times, review their cart, and check order status.

BlossomGifts provides a centralized online platform with categories such as flower bouquets, gift hampers, and personalized gifts. Customers can browse products, add or remove items from the cart, select delivery preferences, and view an order summary. Administrators are responsible for adding, deleting, and updating products and prices, and viewing customer and order information. The supplied code is a front-end demonstration; real login, database inventory, payment processing, email/SMS confirmations, and live tracking require backend services and integrations.

5. Modules

5.1 Admin Module

Admin login and authorized access (planned for the full system).

Add new products and product categories/subcategories.

Remove unavailable or discontinued products.

Modify item prices and product descriptions.

View customer details, checkout information, inventory, and order summaries.

5.2 Customer / User Module

Register and log in (planned for the full system).

Visit the website and browse categories and subcategories.

Search for items, view product details, and check availability.

Add items to the shopping cart, edit quantities, or remove items.

Edit customer information and inquire through email/contact support.

Select delivery date and time, check out, and view order status/history.

Log out and leave the website.

5.3 Shopping Cart and Order Module

Maintain selected products in the cart.

Calculate the total price of cart items.

Allow cart editing and removal of items.

Collect delivery date and time preferences.

Generate an order summary and demo order confirmation.

6. Algorithm

Start the application and display the BlossomGifts home page.

Display popular product categories and their subcategories.

Allow the customer to browse or search products and view item details.

When a product is selected, add it to the shopping cart.

Display cart items and calculate the total amount.

Allow the customer to remove items or continue shopping.

Ask the customer to select a delivery date and time.

Validate that the cart is not empty and a delivery date has been selected.

If validation succeeds, generate a demo order reference and display the order summary.

In a production system, securely process payment, save the order in a database, and send email/SMS confirmation.

Allow customers to view order history/status and allow administrators to maintain inventory.

Stop.

7. Complete HTML, CSS and JavaScript Code

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>BlossomGifts | Online Flower & Gift Shop</title>

<style>

*{box-sizing:border-box} body{margin:0;font-family:Arial,sans-serif;background:#fff8fa;color:#352b30}

header{background:#8e3155;color:white;padding:17px 5%;display:flex;justify-content:space-between;align-items:center;gap:14px;flex-wrap:wrap}

header h1{margin:0;font-size:25px} nav a{color:white;text-decoration:none;margin-left:17px;font-size:14px}

.container{width:min(1100px,92%);margin:25px auto}

.hero{background:linear-gradient(120deg,#9b4268,#e58a9e);color:white;padding:30px;border-radius:16px}

.hero h2{font-size:28px;margin:0 0 8px}.hero p{line-height:1.5;max-width:650px}

button,.btn{border:0;border-radius:8px;padding:11px 15px;background:#8e3155;color:white;font-weight:bold;cursor:pointer}

.btn{display:inline-block;text-decoration:none;background:white;color:#8e3155;margin-top:8px}

h2{color:#81314f}.grid{display:grid;grid-template-columns:repeat(3,1fr);gap:16px;margin-top:18px}

.card,.panel{background:white;border:1px solid #f1dce4;border-radius:13px;padding:18px;box-shadow:0 4px 15px #702b4510}

.flower{height:105px;border-radius:10px;background:#fbe5ec;display:grid;place-items:center;font-size:48px}
```

```
.card h3{color:#81314f;margin:12px 0 5px}.muted{color:#766a70;font-size:14px;line-height:1.5}

.price{font-size:20px;font-weight:bold;color:#8e3155;margin:10px 0}

.actions{display:flex;gap:10px;flex-wrap:wrap;margin-top:12px}

.panel{margin-top:22px} label{display:block;font-size:13px;font-weight:bold;margin:11px 0 5px}

input,select{width:100%;padding:10px;border:1px solid #e5cbd5;border-radius:8px}

.form-row{display:grid;grid-template-columns:1fr 1fr;gap:13px}

table{width:100%;border-collapse:collapse;margin-top:12px;font-size:14px}

th,td{text-align:left;padding:10px;border-bottom:1px solid #f0e1e7}th{color:#81314f;background:#fff6f9}

#message{color:#18764b;font-weight:bold} footer{text-align:center;color:#766a70;padding:25px;font-size:13px}

@media(max-width:720px){.grid,.form-row{grid-template-columns:1fr}nav a{margin:0 10px 0 0}.hero h2{font-size:23px}}

</style>

</head>

<body>

<header><h1>🌸 BlossomGifts</h1><nav><a href="#shop">Shop</a><a href="#cart">Cart</a><a href="#orders">Orders</a></nav></header>

<main class="container">

<section class="hero"><h2>Make every moment bloom.</h2>

<p>Discover flowers, gift baskets and personalized surprises for birthdays, anniversaries and every special occasion. Choose your gift and schedule delivery.</p>

<a class="btn" href="#shop">Explore gifts</a></section>

<section id="shop"><h2>Shop by category</h2><p class="muted">Popular categories and subcategories are shown below.</p>

<div class="grid">

<article class="card"><div class="flower">💐</div><h3>Flower Bouquets</h3><p class="muted">Roses · Lilies · Mixed Flowers</p><div class="price">₹799</div><button onclick="addToCart('Flower Bouquet',799)">Add to cart</button></article>

<article class="card"><div class="flower">🎁</div><h3>Gift Hampers</h3><p class="muted">Chocolate · Fruit · Celebration</p><div class="price">₹999</div><button onclick="addToCart('Gift Hamper',999)">Add to cart</button></article>
```

<article class="card"> 🧸 <h3>Personalized Gifts</h3> Photo Frames · Mugs · Keepsakes ₹499 <button onclick="addToCart('Personalized Gift',499)">Add to cart</button></article>

</section>

<section id="cart" class="panel"><h2>Shopping cart</h2>

<table><thead><tr><th>Item</th><th>Price</th><th>Action</th></tr></thead><tbody id="cartRows"><tr><td colspan="3">Your cart is empty.</td></tr></tbody></table>

Total: ₹0

<label for="deliveryDate">Delivery date</label><input id="deliveryDate" type="date"> <label for="deliveryTime">Delivery time</label><select id="deliveryTime"><option>9 AM - 12 PM</option><option>12 PM - 3 PM</option><option>3 PM - 6 PM</option></select>

<button onclick="checkout()">Checkout</button><button onclick="clearCart()">Clear cart</button> </section>

<section id="orders" class="panel"><h2>Order summary</h2> Demo order history for this browser. <table><thead><tr><th>Order</th><th>Items</th><th>Delivery</th><th>Status</th></tr></thead><tbody id="orderRows"><tr><td colspan="4">No orders placed yet.</td></tr></tbody></table></section>

<section class="panel"><h2>Admin inventory (demo)</h2> In a complete system, authorized administrators can add items, remove items and update prices. This front-end sample displays the customer shopping flow only. </section>

</main><footer>© 2026 BlossomGifts · Student project demonstration</footer>

<script>

let cart=[];

function addToCart(name,price){cart.push({name,price});renderCart();document.getElementById('message').textContent=name+' added to cart.';}

function renderCart(){

const body=document.getElementById('cartRows');body.innerHTML='';let total=0;

if(!cart.length){body.innerHTML='<tr><td colspan="3">Your cart is empty.</td></tr>';}

cart.forEach((item,i)=>{total+=item.price;const tr=document.createElement('tr');

const a=document.createElement('td');a.textContent=item.name;const b=document.createElement('td');b.textContent='₹'+item.price;

```
const c=document.createElement('td');const
btn=document.createElement('button');btn.textContent='Remove';btn.onclick=()=>{cart.splic
e(i,1);renderCart();};c.appendChild(btn);tr.append(a,b,c);body.appendChild(tr);});

document.getElementById('total').textContent='₹'+total;

}

function
clearCart(){cart=[];renderCart();document.getElementById('message').textContent='Cart
cleared.';}

function checkout(){

const msg=document.getElementById('message');const
date=document.getElementById('deliveryDate').value;

if(!cart.length){msg.textContent='Please add an item before checkout.';return;}

if(!date){msg.textContent='Please select a delivery date.';return;}

const summary=cart.map(x=>x.name).join(', ');const tr=document.createElement('tr');

[ 'BG-'+Date.now().toString().slice(-6),summary,date+'
('+document.getElementById('deliveryTime').value+')','Order placed (demo)'
].forEach(v=>{const
td=document.createElement('td');td.textContent=v;tr.appendChild(td);});

const rows=document.getElementById('orderRows');if(rows.textContent.includes('No
orders'))rows.innerHTML='';rows.prepend(tr);

msg.textContent='Demo order placed successfully. No payment or real delivery has been
initiated.';

clearCart();

}

</script>

</body></html>
```

8. Sample Input

Field	Sample value
Product	Flower Bouquet
Price	₹799
Cart quantity	1
Delivery date	20 December 2026
Delivery time	12 PM - 3 PM

Action: Select “Add to cart”, choose a delivery date, and select “Checkout”.

9. Expected Output with Image

The home page displays the BlossomGifts navigation bar, welcome banner, product categories, and shopping cart. When a customer adds an item, the cart displays its name and price and updates the total. The customer can remove items, choose a delivery date and time, and place a demo order. The order summary displays the generated order reference and delivery details.

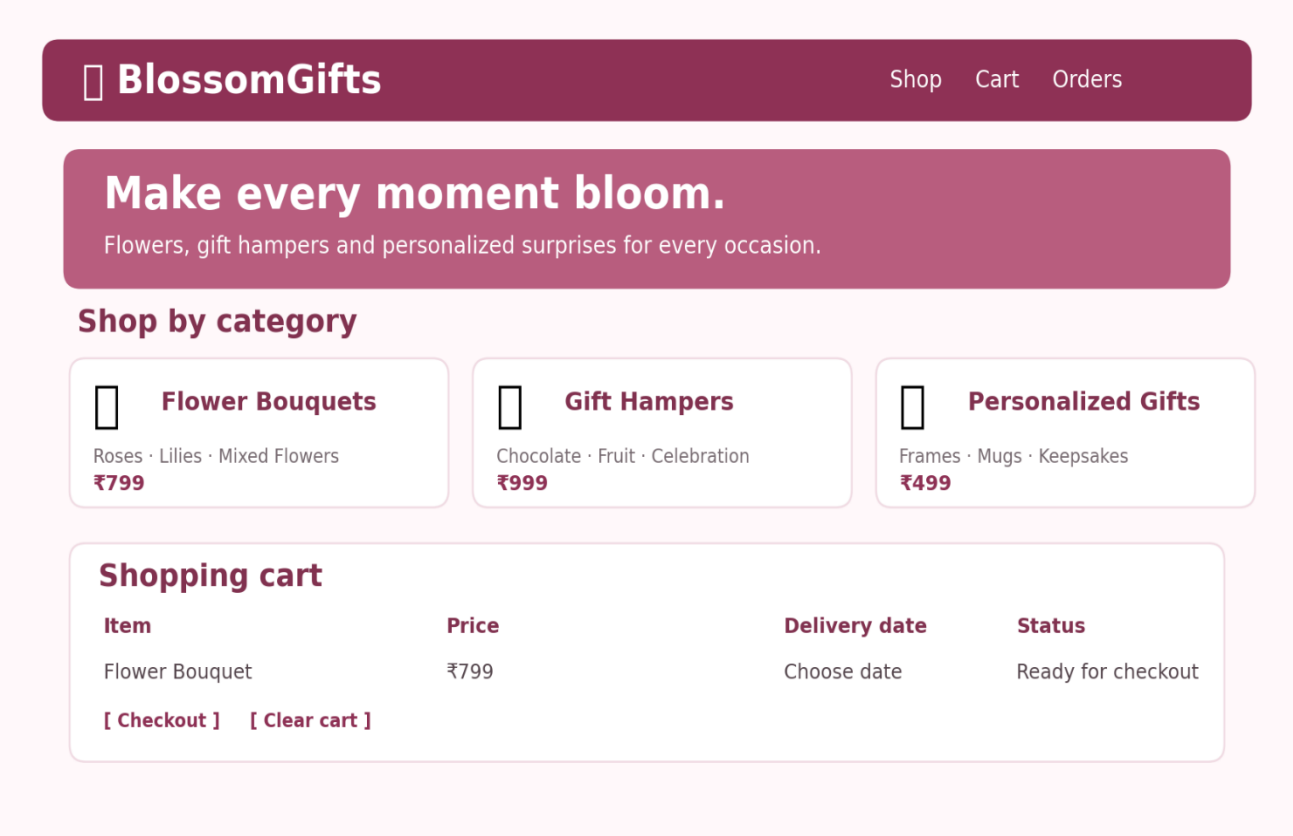


Figure 1: Illustrative BlossomGifts shopping page and cart.

10. Result

The BlossomGifts front-end prototype was developed using HTML, CSS, and JavaScript. It demonstrates product category display, shopping cart operations, total price calculation, delivery date/time selection, and a demo checkout with order summary. The project provides a basic foundation for an online flower and gift ordering system. For production use, it should be extended with secure authentication, database-backed inventory and orders, admin authorization, payment gateway integration, email/SMS notifications, and real-time delivery tracking.

Experiment-14

RENEWMARKET
ONLINE SECOND-HAND GOODS MARKETPLACE

1. Title

ReNewMarket – Online Second-Hand Goods Marketplace

2. Aim

To design and develop an online marketplace that connects buyers and sellers of pre-owned products. The system allows users to browse categories, search listings, view item details, create seller listings, manage shopping activities, and track order status. It also provides administrator functions for managing users, categories, commodities/listings, and orders.

3. Software Requirements

Requirement	Specification
Operating system	Windows 10/11, Linux, or macOS
Front-end	HTML5, CSS3, JavaScript
Browser	Google Chrome, Mozilla Firefox, or Microsoft Edge
Editor	Visual Studio Code / Notepad++
Prototype	Client-side demonstration in a web browser
Production backend	Node.js / PHP / Python with MySQL or equivalent database
Optional integrations	Secure payment gateway, image storage, email/SMS and delivery/logistics APIs

4. Problem Description

Buying and selling second-hand goods through informal channels can be difficult. Buyers may have trouble finding suitable products, comparing prices, checking item information, and communicating with sellers. Sellers may lack a convenient way to publish listings and manage enquiries or orders. Administrators also need tools to maintain user accounts, categories, product information, and order records.

ReNewMarket provides a centralized marketplace for pre-loved electronics, furniture, clothing, and collectibles. Users can browse categories, view available items, create listings, and follow order status. Administrators can manage user accounts, add or delete categories, edit or delete commodities, and update order logistics status to Shipped or Received. The included implementation is a front-end demonstration; secure login, persistent database operations, payments, messaging, and actual logistics tracking require backend integration.

5. Modules

5.1 Admin Module

Admin login using an administrator-defined account and password (authentication must be implemented securely on the server).

Edit user information and delete user accounts when authorized.

Add and delete category information.

Edit commodity/listing details and delete commodities.

View and manage customer orders.

Change logistics status of orders to Shipped or Received, and delete orders when permitted.

5.2 User / Buyer Module

Register and log in to the marketplace (planned for the full system).

Browse categories such as electronics, furniture, clothing, and collectibles.

Search and filter listings by category, price, and other attributes.

View product descriptions, photos, and prices.

Add products to favorites and contact sellers to negotiate prices (planned features).

Place orders, review transaction history, and track order status.

5.3 Seller Module

Create a seller listing with item name, category, description, photos, and price.

Edit or remove own listings (with server-side authorization).

Respond to buyer enquiries and negotiate prices (planned features).

Review received orders and coordinate pickup or delivery (planned features).

5.4 Order Management Module

Create and display order summaries.

Maintain order records and transaction status.

Allow authorized administrators to update logistics status to Shipped or Received.

Delete orders according to administrative rules.

6. Algorithm

Start the application and display the ReNewMarket home page.

Display product categories and available listings.

Allow buyers to search or filter products and view item details.

Allow sellers to enter product details and publish a listing after validation.

Allow a buyer to select an item and proceed to the demo cart/order process.

Generate an order reference and display the order summary.

Allow the administrator to manage user accounts, categories, commodities, and orders.

When an order is dispatched, update its logistics status to Shipped; after delivery, update it to Received.

Display the latest order status to the user.

In a production system, store all changes in a database and use secure authentication, payment, and logistics services.

Stop.

7. Complete HTML, CSS and JavaScript Code

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>ReNewMarket | Pre-loved Marketplace</title>

<style>

*{box-sizing:border-box}body{margin:0;font-family:Arial,sans-serif;background:#f4f8f5;color:#26352c}

header{background:#205b43;color:white;padding:17px 5%;display:flex;justify-content:space-between;align-items:center;gap:14px;flex-wrap:wrap}

header h1{margin:0;font-size:25px}nav a{color:white;text-decoration:none;margin-left:17px;font-size:14px}

.container{width:min(1100px,92%);margin:25px auto}.hero{background:linear-gradient(120deg,#24694c,#7eaa72);color:white;padding:30px;border-radius:16px}

.hero h2{font-size:28px;margin:0 0 8px}.hero p{line-height:1.5;max-width:650px}

button,.btn{border:0;border-radius:8px;padding:11px 15px;background:#24694c;color:white;font-weight:bold;cursor:pointer}

.btn{display:inline-block;text-decoration:none;background:white;color:#205b43;margin-top:8px}

h2{color:#205b43}.grid{display:grid;grid-template-columns:repeat(3,1fr);gap:16px;margin-top:18px}

.card,.panel{background:white;border:1px solid #dce9df;border-radius:13px;padding:18px;box-shadow:0 4px 15px #244b3210}

```
.product-art{height:105px;border-radius:10px;background:#e8f2e9;display:grid;place-items:center;font-size:48px}

.card h3{color:#205b43;margin:12px 0 5px}.muted{color:#66756a;font-size:14px;line-height:1.5}

.price{font-size:20px;font-weight:bold;color:#205b43;margin:10px 0}.actions{display:flex;gap:10px;flex-wrap:wrap;margin-top:12px}

.panel{margin-top:22px}label{display:block;font-size:13px;font-weight:bold;margin:11px 0 5px}

input,select,textarea{width:100%;padding:10px;border:1px solid #cbd9ce;border-radius:8px}

.form-row{display:grid;grid-template-columns:1fr 1fr;gap:13px}

table{width:100%;border-collapse:collapse;margin-top:12px;font-size:14px}th,td{text-align:left;padding:10px;border-bottom:1px solid #e5eee7}

th{color:#205b43;background:#f4f8f5}#message{color:#18764b;font-weight:bold}

footer{text-align:center;color:#66756a;padding:25px;font-size:13px}

@media(max-width:720px){.grid,.form-row{grid-template-columns:1fr}nav a{margin:0 10px 0 0}.hero h2{font-size:23px}}

</style></head>

<body>

<header><h1>♻ ReNewMarket</h1><nav><a href="#listings">Browse</a><a href="#sell">Sell an item</a><a href="#orders">Orders</a></nav></header>

<main class="container">

<section class="hero"><h2>Give pre-loved items a new life.</h2><p>Discover quality second-hand electronics, furniture, clothing and collectibles. Buy, sell and reuse with confidence.</p><a class="btn" href="#listings">Explore listings</a></section>

<section id="listings"><h2>Browse popular categories</h2><div class="grid">

<article class="card"><div class="product-art">📱</div><h3>Electronics</h3><p class="muted">Phones · Laptops · Accessories</p><div class="price">From ₹2,500</div><button onclick="addItem('Used Smartphone',2500)">Add to cart</button></article>

<article class="card"><div class="product-art">🪑</div><h3>Furniture</h3><p class="muted">Chairs · Tables · Storage</p><div class="price">From ₹1,800</div><button onclick="addItem('Study Chair',1800)">Add to cart</button></article>

<article class="card"><div class="product-art">👕</div><h3>Clothing</h3><p class="muted">Men · Women · Kids</p><div class="price">From ₹300</div><button onclick="addItem('Pre-loved Jacket',300)">Add to cart</button></article>

</div></section>
```

<section id="sell" class="panel"><h2>Create a seller listing</h2>

<label for="itemName">Item name</label><input id="itemName" placeholder="e.g. Wooden study table">

<label for="itemPrice">Price (₹)</label><input id="itemPrice" type="number" min="1" placeholder="Enter price">

<label for="itemCategory">Category</label><select id="itemCategory"><option>Electronics</option><option>Furniture</option><option>Clothing</option><option>Collectibles</option></select>

<button onclick="createListing()">Publish demo listing</button>

<table><thead><tr><th>Seller listing</th><th>Category</th><th>Price</th></tr></thead><tbody id="listingRows"><tr><td>Vintage Camera</td><td>Collectibles</td><td>₹3,200</td></tr></tbody></table></section>

<section class="panel"><h2>Shopping cart</h2><table><thead><tr><th>Item</th><th>Price</th><th>Action</th></tr></thead><tbody id="cartRows"><tr><td colspan="3">Your cart is empty.</td></tr></tbody></table>

Total: ₹0 <button onclick="checkout()">Place demo order</button><button onclick="clearCart()">Clear cart</button> </section>

<section id="orders" class="panel"><h2>Order management</h2> Demo order status can be changed to Shipped or Received.

<table><thead><tr><th>Order ID</th><th>Items</th><th>Status</th><th>Update</th></tr></thead><tbody id="orderRows"><tr><td colspan="4">No orders yet.</td></tr></tbody></table></section>

<section class="panel"><h2>Administrator functions</h2> A full system administrator can manage user accounts, categories, commodities/listings, and orders. Secure admin authentication and persistent database operations are not included in this front-end demo. </section>

</main><footer>© 2026 ReNewMarket · Student project demonstration</footer>

<script>

let cart=[],orders=[],listingCount=0;

function addItem(name,price){cart.push({name,price});renderCart();document.getElementById('message').textContent=name+' added to cart.';}

function renderCart(){const body=document.getElementById('cartRows');body.innerHTML='';let total=0;

if(!cart.length)body.innerHTML='<tr><td colspan="3">Your cart is empty.</td></tr>';

```
cart.forEach((item,i)=>{total+=item.price;const tr=document.createElement('tr');

[item.name,'₹'+item.price].forEach(v=>{const
td=document.createElement('td');td.textContent=v;tr.appendChild(td);});

const
td=document.createElement('td'),b=document.createElement('button');b.textContent='Remov
e';b.onclick=()=>{cart.splice(i,1);renderCart();};td.appendChild(b);tr.appendChild(td);body.a
ppendChild(tr);});

document.getElementById('total').textContent='₹'+total;}

function
clearCart(){cart=[];renderCart();document.getElementById('message').textContent='Cart
cleared.';}

function createListing(){const
name=document.getElementById('itemName').value.trim(),price=Number(document.getEle
mentById('itemPrice').value),cat=document.getElementById('itemCategory').value,msg=doc
ument.getElementById('listingMessage');

if(!name||!Number.isFinite(price)||price<=0){msg.textContent='Enter an item name and a
valid price.';return;}

const tr=document.createElement('tr');[name,cat,'₹'+price].forEach(v=>{const
td=document.createElement('td');td.textContent=v;tr.appendChild(td);});document.getElemen
tById('listingRows').prepend(tr);

msg.textContent='Demo listing
published.';document.getElementById('itemName').value='';document.getElementById('item
Price').value='';}

function checkout(){const
msg=document.getElementById('message');if(!cart.length){msg.textContent='Add an item
before placing an order.';return;}

const order={id:'RNM-'+Date.now().toString().slice(-6),items:cart.map(x=>x.name).join(',
'),status:'Placed'};

orders.push(order);const
body=document.getElementById('orderRows');if(body.textContent.includes('No
orders'))body.innerHTML='';const tr=document.createElement('tr');

const id=document.createElement('td');id.textContent=order.id;const
items=document.createElement('td');items.textContent=order.items;const
status=document.createElement('td');status.textContent=order.status;

const
action=document.createElement('td'),select=document.createElement('select');['Shipped','Rec
eived'].forEach(s=>{const
o=document.createElement('option');o.textContent=s;select.appendChild(o);});
```

select.onchange=()=>{status.textContent=select.value;};action.appendChild(select);tr.append(id,items,status,action);body.prepend(tr);

msg.textContent='Demo order created. No payment or real shipment has been initiated.';clearCart();}

</script></body></html>

8. Sample Input

Field	Sample value
Listing name	Used Smartphone
Category	Electronics
Price	₹2,500
Buyer action	Add to cart and place demo order
Order status update	Shipped, then Received

Action: Add a sample product to the cart, place a demo order, and select Shipped or Received from the order management table.

9. Expected Output with Image

The home page displays the ReNewMarket navigation bar, welcome banner, and popular product categories. Sellers can enter an item name, price, and category to publish a demo listing. Buyers can add products to the cart and create a demo order. The order management table displays an order reference and allows its status to be changed to Shipped or Received.

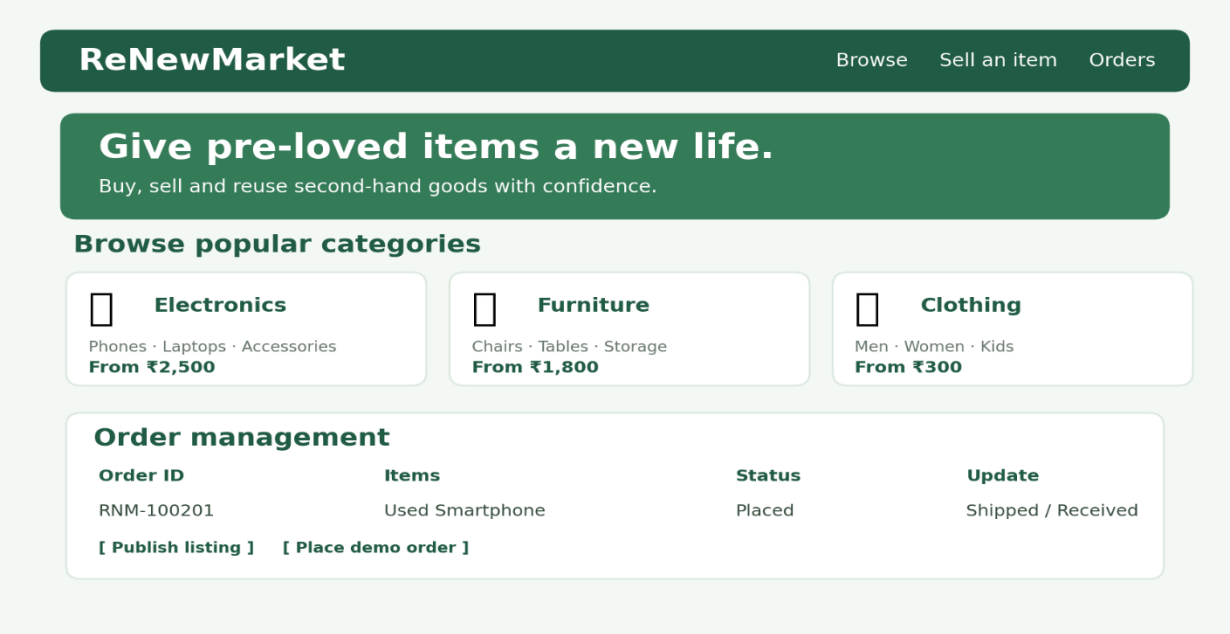


Figure 1: Illustrative ReNewMarket marketplace and order management screen.

10. Result

The ReNewMarket front-end prototype was developed using HTML, CSS, and JavaScript. It demonstrates product category display, seller listing creation, shopping cart operations, demo order generation, and order status updates. The project provides a foundation for a second-hand goods marketplace. For real-world deployment, it should be extended with secure user and administrator authentication, database-backed user/category/product/order management, image uploads, buyer-seller communication, secure payments, and delivery tracking.

Experiment-15

EASYTOLL
ONLINE ADVANCE TOLL PAYMENT SYSTEM

1. Title

EasyToll – Online Advance Toll Payment System

2. Aim

To design and develop a web-based toll payment system that enables users to manage vehicle details, add money to an eWallet, purchase toll e-tickets, apply for or renew toll passes, and access QR-based pass or ticket identification. The system also provides administrator functions for managing users and toll details and toll-gate operator functions for verifying passes and updating toll locations.

3. Software Requirements

Requirement	Specification
Operating system	Windows 10/11, Linux, or macOS
Front-end	HTML5, CSS3, JavaScript
Browser	Google Chrome, Mozilla Firefox, or Microsoft Edge
Editor	Visual Studio Code / Notepad++
Prototype	Client-side web application demonstration
Production backend	Node.js / PHP / Python with MySQL or equivalent database
External integrations	Payment gateway, QR-code library, notification service, and toll-system APIs

4. Problem Description

Traditional toll collection can cause queues and delays, especially during busy travel periods. Drivers may also find it difficult to keep track of toll balances, payments, vehicle details, and

pass renewals. A convenient advance-payment platform can help users prepare toll payments before reaching a toll gate and reduce manual payment steps.

EasyToll proposes an online system where users can register, maintain vehicle details, add money to an eWallet, buy toll e-tickets, apply for or renew toll passes, and view transaction history. Administrators can manage users and toll-pass information, manage toll login accounts, approve passes, generate QR codes, and create toll details. Toll-gate operators can log in, scan or look up QR identifiers, verify pass or ticket IDs, and update toll location information. The provided implementation is a front-end demonstration only; actual payment, secure QR validation, account authentication, and toll-gate integration require server-side services.

5. Modules

5.1 Admin Module

Admin login with secure role-based access (to be implemented on the backend).

Manage user accounts and toll-pass records.

Manage toll-gate login accounts.

Approve or reject toll-pass applications.

Generate QR codes for approved toll passes and tickets.

Create and maintain toll details such as toll-gate name, location, and fee.

5.2 User Module

User registration and login.

Add money to the eWallet and view the available balance.

Buy toll e-tickets for selected routes.

Apply for a new toll pass and renew an existing pass.

View or obtain the QR code associated with a toll pass.

Manage vehicle details and review transaction history.

5.3 Toll Gate User Module

Toll-gate operator login.

Scan a QR code or enter a pass/ticket identifier.

Verify toll pass ID and toll ticket validity with the server.

Update the toll-gate location or transaction checkpoint.

Record successful passage and prevent duplicate or invalid use (production feature).

6. Algorithm

Start the EasyToll application and display the home page.

Allow the user to register or log in (full-system feature).

Display wallet balance and registered vehicle information.

Allow the user to add money to the eWallet and update the displayed demo balance.

Allow the user to select a toll route and ticket type.

Check whether the wallet balance is sufficient for the selected toll ticket.

If the balance is insufficient, display a message asking the user to add money.

If sufficient, deduct the demo ticket fee, create a ticket ID, and display an illustrative QR placeholder.

Allow the user to apply for or renew a toll pass; send the application to the administrator for approval in the full system.

Allow the toll-gate operator to enter or scan the ticket/pass ID and verify it against server records.

Update the toll location and record the transaction in the full system.

Display the transaction result and stop.

7. Complete HTML, CSS and JavaScript Code

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>EasyToll | Advance Toll Payment</title>

<style>

*{box-sizing:border-box}body{margin:0;font-family:Arial,sans-serif;background:#f3f7fb;color:#263445}

header{background:#153e68;color:white;padding:17px 5%;display:flex;justify-content:space-between;align-items:center;gap:14px;flex-wrap:wrap}

header h1{margin:0;font-size:25px}nav a{color:white;text-decoration:none;margin-left:17px;font-size:14px}

.container{width:min(1100px,92%);margin:25px auto}.hero{background:linear-gradient(120deg,#174c7e,#1595a3);color:white;padding:30px;border-radius:16px}

.hero h2{font-size:28px;margin:0 0 8px}.hero p{line-height:1.5;max-width:680px}

button,.btn{border:0;border-radius:8px;padding:11px 15px;background:#f3c85b;color:#193b5c;font-weight:bold;cursor:pointer}

.btn{display:inline-block;text-decoration:none;margin-top:8px}.grid{display:grid;grid-template-columns:repeat(3,1fr);gap:16px;margin-top:18px}

```
.card,.panel{background:white;border:1px solid #dce6ef;border-radius:13px;padding:18px;box-shadow:0 4px 15px #173b5b10}

.card h3{color:#174c7e;margin:5px 0}.muted{color:#667789;font-size:14px;line-height:1.5}

.amount{font-size:25px;font-weight:bold;color:#174c7e;margin:8px 0}

.panel{margin-top:22px}h2{color:#174c7e}label{display:block;font-size:13px;font-weight:bold;margin:11px 0 5px}

input,select{width:100%;padding:10px;border:1px solid #cbd8e4;border-radius:8px}

.form-row{display:grid;grid-template-columns:1fr 1fr;gap:13px}.actions{display:flex;gap:10px;flex-wrap:wrap;margin-top:12px}

table{width:100%;border-collapse:collapse;margin-top:12px;font-size:14px}th,td{text-align:left;padding:10px;border-bottom:1px solid #e5edf4}

th{color:#174c7e;background:#f5f8fb}#message{color:#18764b;font-weight:bold}

.qr{display:inline-block;background:#fff;border:6px solid #edf2f7;padding:12px;font-family:monospace;font-size:16px;letter-spacing:3px;color:#173b5b;margin-top:10px}

footer{text-align:center;color:#667789;padding:25px;font-size:13px}

@media(max-width:720px){.grid,.form-row{grid-template-columns:1fr}nav a{margin:0 10px 0 0}.hero h2{font-size:23px}}

</style></head>

<body>

<header><h1>EasyToll</h1><nav><a href="#account">Account</a><a href="#passes">Toll Pass</a><a href="#gate">Toll Gate</a></nav></header>

<main class="container">

<section class="hero"><h2>Pay tolls in advance. Travel without waiting.</h2><p>Manage your vehicle, add wallet balance, purchase a demo e-ticket and view your toll pass details from one convenient dashboard.</p><a class="btn" href="#account">Manage my account</a></section>

<section class="grid">

<article class="card"><h3>eWallet Balance</h3><div class="amount" id="balance">₹500</div><p class="muted">Available demo wallet balance</p></article>

<article class="card"><h3>Vehicle</h3><div class="amount">AP XX 1234</div><p class="muted">Sample registered vehicle</p></article>

<article class="card"><h3>Toll Pass</h3><div class="amount">Active</div><p class="muted">Demo pass status</p></article>

</section>
```

<section id="account" class="panel"><h2>Wallet and vehicle details</h2>

<label for="walletAmount">Add money (₹)</label><input id="walletAmount" type="number" min="1" placeholder="Enter amount">

<label for="vehicle">Vehicle number</label><input id="vehicle" value="AP XX 1234">

<button onclick="addMoney()">Add money to wallet</button><button onclick="saveVehicle()">Update vehicle</button> </section>

<section id="passes" class="panel"><h2>Buy toll e-ticket / apply for pass</h2>

<label for="route">Toll route</label><select id="route"><option>City A – City B</option><option>City B – City C</option><option>City A – City C</option></select>

<label for="ticketType">Ticket type</label><select id="ticketType"><option value="120">Single trip – ₹120</option><option value="200">Return trip – ₹200</option><option value="500">Monthly pass – ₹500</option></select>

<button onclick="buyTicket()">Buy demo e-ticket</button><button onclick="applyPass()">Apply / renew toll pass</button>

</section>

<section id="gate" class="panel"><h2>Toll gate verification</h2> Demo toll-gate operator can enter a pass or ticket ID to verify it.

<label for="passId">Pass / ticket ID</label><input id="passId" placeholder="Enter generated ID">

<label for="location">Toll gate location</label><input id="location" placeholder="Enter toll gate location">

<button onclick="verifyPass()">Verify pass / ticket</button><button onclick="updateLocation()">Update toll location</button> </section>

<section class="panel"><h2>Transaction history</h2><table><thead><tr><th>Transaction</th><th>Details</th><th>Amount</th><th>Status</th></tr></thead><tbody id="history"><tr><td colspan="4">No transactions yet.</td></tr></tbody></table></section>

<section class="panel"><h2>Admin functions (system scope)</h2> The complete system administrator can manage users and toll passes, manage toll login accounts, approve passes, generate QR codes, and create toll details. Secure role-based access and persistent database operations must be implemented on the backend. </section>

</main><footer>© 2026 EasyToll · Internet Programming Lab Experiment</footer>

```
<script>

let wallet=500,validIds=[];

function addHistory(details,amount,status){const
body=document.getElementById('history');if(body.textContent.includes('No
transactions'))body.innerHTML='';

const tr=document.createElement('tr');['TXN-'+Date.now().toString().slice(-
6),details,'₹'+amount,status].forEach(v=>{const
td=document.createElement('td');td.textContent=v;tr.appendChild(td);});body.prepend(tr);}

function addMoney(){const
amt=Number(document.getElementById('walletAmount').value),msg=document.getElement
ById('message');

if(!Number.isFinite(amt)||amt<=0){msg.textContent='Enter a valid
amount.';return;}wallet+=amt;document.getElementById('balance').textContent='₹'+wallet;

addHistory('Wallet top-up',amt,'Demo success');msg.textContent='Money added to demo
wallet.';document.getElementById('walletAmount').value='';}

function saveVehicle(){const
v=document.getElementById('vehicle').value.trim();document.getElementById('message').te
xtContent=v?'Vehicle details updated (demo).':'Enter a vehicle number.';}

function makeId(){return 'ET-'+Math.floor(100000+Math.random()*900000);}

function buyTicket(){const
fee=Number(document.getElementById('ticketType').value),route=document.getElementById
d('route').value,msg=document.getElementById('message');

if(wallet<fee){msg.textContent='Insufficient wallet balance. Please add
money.';return;}wallet-=fee;document.getElementById('balance').textContent='₹'+wallet;

const id=makeId();validIds.push(id);document.getElementById('passId').value=id;

document.getElementById('qrArea').innerHTML='<h3>Your demo e-ticket</h3><p>Ticket
ID: <strong>'+id+'</strong><br>Route: '+route+'</p><div class="qr">▦ '+id+' ▦</div><p
class="muted">Illustrative QR placeholder only; not a scannable or payment credential.</p>';

addHistory('Toll ticket: '+route,fee,'Purchased (demo)');msg.textContent='Demo ticket
purchased and wallet updated.';}

function applyPass(){const
id=makeId();validIds.push(id);document.getElementById('passId').value=id;

document.getElementById('qrArea').innerHTML='<h3>Demo toll pass
application</h3><p>Application ID: <strong>'+id+'</strong></p><div class="qr">▦ '+id+'
▦</div><p class="muted">Application is a front-end demo; administrator approval is not
connected.</p>';

document.getElementById('message').textContent='Demo toll pass application created.';}
```

function verifyPass(){const id=document.getElementById('passId').value.trim();document.getElementById('gateMessage').textContent=validIds.includes(id)?'Verified in this demo session.':'ID not found in this demo session.';}

function updateLocation(){const loc=document.getElementById('location').value.trim();document.getElementById('gateMessage').textContent=loc?'Toll gate location updated (demo): '+loc:'Enter a toll gate location.';}

</script></body></html>

8. Sample Input

Field	Sample value
Vehicle number	AP XX 1234
Initial eWallet balance	₹500
Amount to add	₹300
Toll route	City A – City B
Ticket type	Single trip – ₹120
Toll gate location	Highway Toll Plaza

Action: Add ₹300 to the wallet, select a route and single-trip ticket, buy the demo e-ticket, and verify the generated ticket ID in the toll-gate section.

9. Expected Output with Image

The EasyToll dashboard displays the wallet balance, vehicle information, and toll-pass status. After adding money, the balance increases. When a user purchases a ticket, the fare is deducted and a demo ticket ID with an illustrative QR placeholder is displayed. The toll-gate section allows the operator to verify an ID created during the current browser session and enter a toll location.

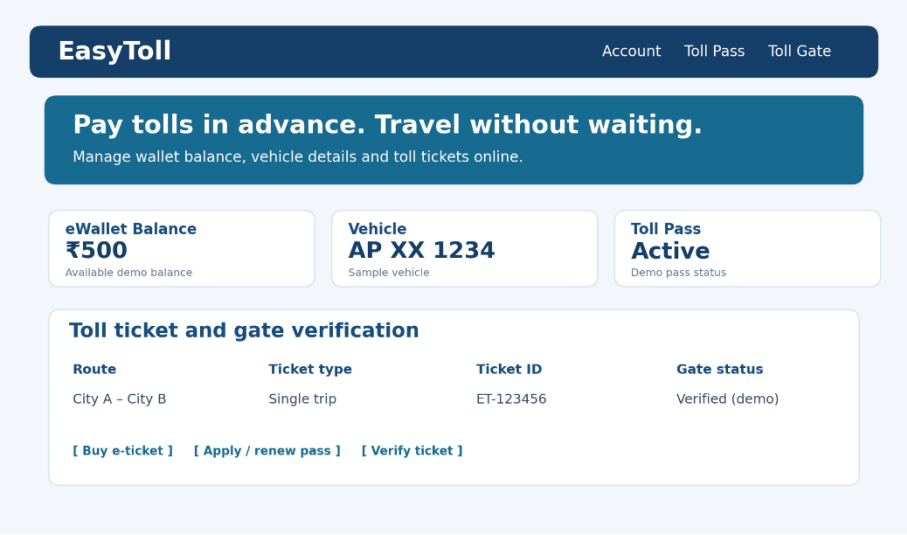


Figure 1: Illustrative EasyToll dashboard and toll ticket interface.

10. Result

The EasyToll front-end prototype was developed using HTML, CSS, and JavaScript. It demonstrates eWallet top-up, vehicle detail updates, toll e-ticket purchase, pass application, ticket identification, demo verification, and toll-location updates. The project provides a foundation for an online advance toll payment system intended to reduce payment delays at toll booths. For real-world use, it must be extended with secure authentication, database storage, payment-gateway integration, genuine QR generation and server-side verification, administrator approval workflows, and integration with toll-gate equipment.

Experiment-16

COMPLAINEASE – ONLINE COMPLAINT MANAGEMENT SYSTEM

1. Title

ComplainEase – Online Complaint Registration and Management System

2. Aim

To design and develop a web-based complaint management system that enables users to register complaints, mark complaint locations, track status and submit feedback, while allowing officers to investigate complaints and update progress and administrators to manage accounts, access rights and portal information.

3. Software Requirements

Operating System: Windows, Linux or macOS

Frontend: HTML5, CSS3 and JavaScript

Development Environment: Visual Studio Code or any text editor

Web Browser: Google Chrome, Mozilla Firefox or Microsoft Edge

Storage for this lab prototype: Browser Local Storage

Optional production services: Backend server, database, email/SMS gateway and Google Maps API

4. Problem Description

Traditional complaint handling may involve paper forms, repeated office visits and limited visibility into the progress of an issue. Citizens may not know whether a complaint has been received, assigned or resolved. ComplainEase provides a centralized online portal where a user can register and log in, submit a complaint with category, description, supporting evidence and location, view complaint history and status, give feedback and obtain administrator contact details. Officers can register/login, view assigned complaints, update their status and upload proof of work. Administrators have full system control, including account creation/deletion, officer credential generation, password changes, account viewing, feature visibility settings and portal information management.

5. Modules

A. User Module

User registration and login.

Post a complaint with category, detailed description and supporting document.

Mark the complaint location using a map/location field.

View complaint history and real-time status updates.

Provide feedback and additional evidence.

View administrator contact details.

B. Officer Module

Officer registration and login.

View complaints assigned to the officer.

Investigate complaints and update status (Submitted, In Progress or Resolved).

Update proof of work and resolution remarks.

C. Admin Module

Generate officer IDs and passwords and manage officer accounts.

Create, delete and view user/officer accounts.

Change passwords and control account access.

Hide or enable selected features for users and officers.

Insert and maintain information available on the portal.

Access and manage all officer accounts and oversee complaint resolution.

6. Algorithm

Start the ComplainEase application and display the home page.

Allow the user, officer or administrator to register/login with the appropriate role.

For a user, collect complaint category, description, supporting document and location.

Validate the required fields and generate a unique complaint ID.

Save the complaint and display the confirmation and tracking ID.

Allow the user to view complaint status, history, admin contact details and submit feedback.

Allow the officer to view assigned complaints, investigate them, update status and attach proof of work.

Allow the administrator to manage accounts, credentials, passwords, portal information and feature visibility.

Notify the user of important status changes through configured email/SMS services.

Stop.

7. Complete HTML, CSS and JavaScript Code

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>ComplainEase - Complaint Management</title>

<style>

:root{--navy:#172554;--blue:#2563eb;--cyan:#0ea5e9;--bg:#f1f5f9;--ink:#172033;--muted:#64748b;--line:#dbe3ef;--green:#15803d;--amber:#b45309}

*{box-sizing:border-box}body{margin:0;background:var(--bg);font:15px/1.5 Arial,sans-serif;color:var(--ink)}

header{background:linear-gradient(110deg,var(--navy),#1d4ed8);color:white;padding:22px max(5vw,20px);display:flex;justify-content:space-between;align-items:center;gap:16px;flex-wrap:wrap}

.brand{font-size:24px;font-weight:800;letter-spacing:.2px}.brand span{color:#7dd3fc}

header small{color:#dbeafe}main{max-width:1180px;margin:26px auto;padding:0 18px}

.welcome{display:flex;justify-content:space-between;align-items:center;gap:20px;flex-wrap:wrap;margin-bottom:20px}

h1{font-size:27px;margin:0 0 5px}.muted{color:var(--muted)}

.grid{display:grid;grid-template-columns:repeat(4,1fr);gap:14px;margin:18px 0}

.stat,.panel,.card{background:white;border:1px solid var(--line);border-radius:14px;padding:17px;box-shadow:0 3px 12px #0f172a08}

.stat strong{display:block;font-size:26px;color:var(--navy)}.stat span{font-size:13px;color:var(--muted)}

.layout{display:grid;grid-template-columns:1.1fr .9fr;gap:18px}

h2{font-size:18px;margin:0 0 14px;color:var(--navy)}

label{display:block;font-weight:700;font-size:13px;margin:10px 0 5px}

input,select,textarea{width:100%;padding:10px 11px;border:1px solid #cbd5e1;border-radius:8px;font:inherit;background:white}

textarea{min-height:85px;resize:vertical}

```
button{border:0;border-radius:8px;padding:10px 15px;background:var(--blue);color:white;font-weight:700;cursor:pointer;margin-top:12px}

button.secondary{background:#e2e8f0;color:#1e293b}button.danger{background:#fee2e2;color:#991b1b}

button:hover{filter:brightness(.96)}.row{display:grid;grid-template-columns:1fr 1fr;gap:10px}

table{width:100%;border-collapse:collapse;font-size:13px}th,td{text-align:left;padding:10px 8px;border-bottom:1px solid var(--line)}th{color:var(--muted);font-size:12px}

.pill{display:inline-block;padding:3px 9px;border-radius:20px;background:#dbeafe;color:#1d4ed8;font-size:11px;font-weight:700}

.pill.resolved{background:#dcfce7;color:#166534}.pill.progress{background:#fef3c7;color:#92400e}

.note{background:#eff6ff;border-left:4px solid var(--blue);padding:11px;border-radius:6px;font-size:12px;color:#1e40af;margin-top:12px}

.map{height:115px;background:linear-gradient(135deg,#e0f2fe,#dcfce7);border-radius:10px;display:grid;place-items:center;color:#166534;text-align:center;font-weight:700;margin-top:10px;border:1px dashed #86bca0}

.actions{display:flex;gap:8px;flex-wrap:wrap}.actions button{margin-top:0}

@media(max-width:850px){.grid{grid-template-columns:repeat(2,1fr)}.layout{grid-template-columns:1fr}}

@media(max-width:500px){.grid,.row{grid-template-columns:1fr}}

</style></head>

<body>

<header><div><div class="brand">Complain<span>Ease</span></div><small>Complaint registration & resolution portal</small></div><div>Demo portal · User dashboard</div></header>

<main>

<div class="welcome"><div><h1>Good morning, Priya 👋</h1><div class="muted">Track issues, submit complaints and share feedback.</div></div><div class="actions"><button class="secondary" onclick="showAdmin()">Admin contact</button><button onclick="document.getElementById('complaintForm').scrollIntoView({behavior:'smooth'})">＋ New complaint</button></div></div>

<div class="grid">

<div class="stat"><strong id="total">3</strong><span>Total complaints</span></div>

<div class="stat"><strong id="open">1</strong><span>Open</span></div>

<div class="stat"><strong id="progress">1</strong><span>In progress</span></div>
```

1Resolved

<section class="panel" id="complaintForm"><h2>Post a complaint</h2>

<form onsubmit="addComplaint(event)">

<label for="category">Category</label><select id="category" required><option value="">Select category</option><option>Road & Transport</option><option>Water Supply</option><option>Electricity</option><option>Sanitation</option><option>Public Safety</option><option>Other</option></select>

<label for="location">Location / landmark</label><input id="location" required placeholder="e.g. Main Road, Ward 4">

<label for="description">Complaint description</label><textarea id="description" required placeholder="Describe the issue clearly..."></textarea>

<label for="proof">Supporting document (optional)</label><input id="proof" type="file" accept="image/*,.pdf"> <label for="maploc">Map location</label><input id="maploc" placeholder="Enter location or coordinates">

📍 Map location preview
<small>Demo placeholder — connect Google Maps API for live map pinning</small>

<button type="submit">Submit complaint</button>

</form> This browser demo stores complaint records in local storage. Uploaded files are not transmitted or permanently stored.

</section>

<section class="panel"><h2>Complaint tracking</h2> <table><thead><tr><th>ID / Issue</th><th>Status</th><th>Officer</th></tr></thead><tbody id="complaintRows">

<tr><td>CE-1024
Streetlight not working</td><td>In progress</td><td>R. Kumar</td></tr>

<tr><td>CE-1023
Water leakage</td><td>Resolved</td><td>S. Devi</td></tr>

<tr><td>CE-1022
Garbage collection delay</td><td>Submitted</td><td>Pending</td></tr>

</tbody></table>

<h2 style="margin-top:22px">Feedback</h2><label for="feedbackId">Complaint ID</label><input id="feedbackId" placeholder="e.g. CE-1023">

<label for="feedback">Your feedback</label><textarea id="feedback" placeholder="Share your experience..."></textarea>

<button onclick="sendFeedback()">Submit feedback</button>

</section>

<section class="panel" style="margin-top:18px"><h2>Officer / Admin workspace preview</h2>

Officer actions View assigned complaints, update status, add proof of work and resolution notes. <select id="demoStatus"><option>Submitted</option><option>In progress</option><option>Resolved</option></select><button class="secondary" onclick="alert('Demo: status update saved for the selected complaint.')">Update status</button>

Administrator actions Create or delete accounts, generate officer credentials, change passwords, manage portal information and enable or hide features. <button class="secondary" onclick="showAdmin()">View admin contact</button>

</section>

</main>

<script>

let complaints = JSON.parse(localStorage.getItem('ce_complaints') || '[]');

function render(){const rows=document.getElementById('complaintRows');complaints.forEach(c=>{if(!document.getElementById(c.id)){let tr=document.createElement('tr');tr.id=c.id;tr.innerHTML=`<td><b>${c.id}</b><br>${escapeHtml(c.category)}<br><small>${escapeHtml(c.location)}</small></td><td><span class="pill">Submitted</span></td><td>Pending</td>`;rows.prepend(tr)}});document.getElementById('total').textContent=3+complaints.length;document.getElementById('open').textContent=1+complaints.length}

function escapeHtml(s){return String(s).replace(/[&<>"']/g,c=>({'&':'&','<':'<','>':'>','"':'"',"'":'''}[c]))}

function addComplaint(e){e.preventDefault();let id='CE-'+(1025+complaints.length);let c={id,category:document.getElementById('category').value,location:document.getElementById('location').value,description:document.getElementById('description').value};complaints.push(c);localStorage.setItem('ce_complaints',JSON.stringify(complaints));render();alert('Complaint submitted. Your tracking ID is '+id);e.target.reset()}

function sendFeedback(){let id=document.getElementById('feedbackId').value.trim(),msg=document.getElementById('feedback').value.trim();if(!id||!msg){alert('Enter complaint ID and

feedback.');return}document.getElementById('feedbackMsg').textContent='Thank you. Feedback recorded in this demo for '+id+'.';document.getElementById('feedback').value=''}

function showAdmin(){alert('Demo admin contact: support@complainease.example | Helpline: 1800-000-000')}

render();

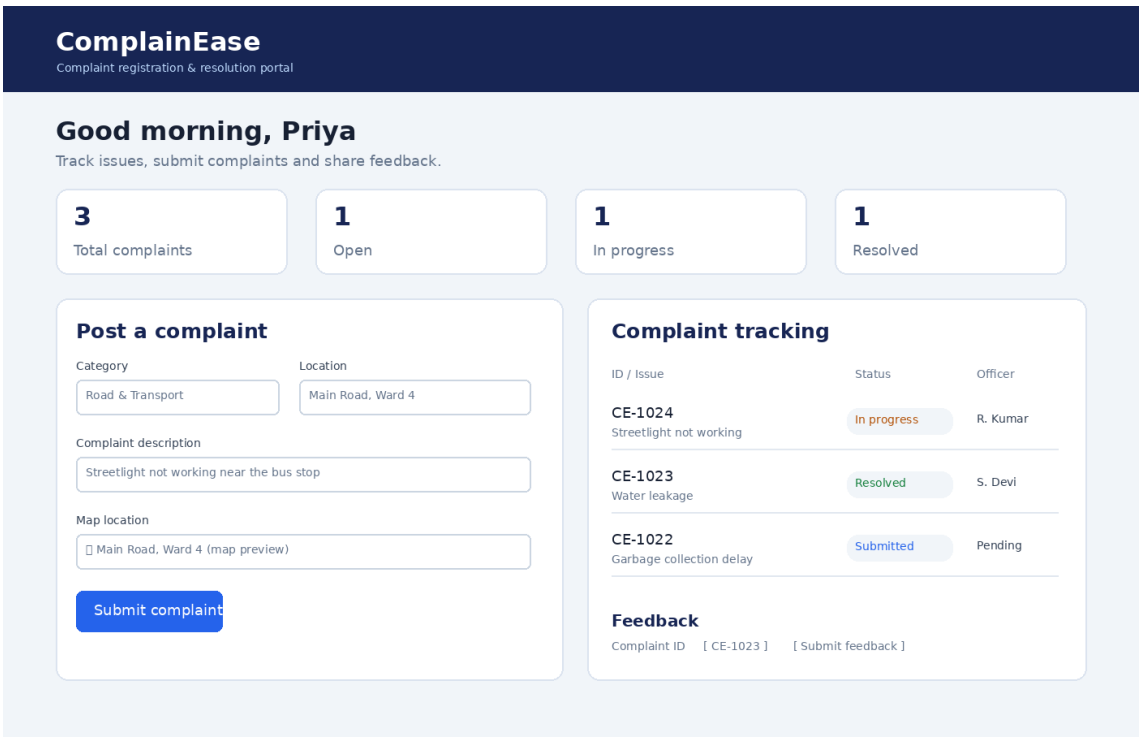
</script></body></html>

8. Sample Input

Field	Sample value
User name	Priya
Category	Road & Transport
Location	Main Road, Ward 4
Description	Streetlight not working near the bus stop.
Supporting document	streetlight_photo.jpg (optional)
Map location	Main Road, Ward 4
Feedback ID	CE-1023
Feedback	The issue was resolved promptly.

9. Expected Output with Image

The dashboard displays complaint summary cards, a complaint submission form with location field, complaint tracking table with status and officer details, and a feedback area. The map preview in this standalone prototype is illustrative; live Google Maps pinning requires a valid Maps API key and integration.



10. Result

Thus, the ComplainEase online complaint management system prototype was designed and implemented using HTML, CSS and JavaScript. It demonstrates complaint registration, tracking, feedback and role-based workflow concepts for users, officers and administrators. For deployment, server-side authentication and authorization, database storage, secure document uploads, actual email/SMS notifications, Google Maps integration and audit logging must be implemented. The browser-only prototype is intended for lab demonstration and is not a production complaint service.

Experiment 17

VOTEONLINE – SECURE E-VOTING PLATFORM

1. Title

VoteOnline – Online Electronic Voting System

2. Aim

To design and develop a web-based e-voting platform that allows registered voters to authenticate using a Voter ID and OTP, view election candidates, select and submit a vote, and receive confirmation. The administrator can manage elections, candidates, wards and voter lists, and view election results.

3. Software Requirements

Operating System: Windows, Linux or macOS

Frontend: HTML5, CSS3 and JavaScript

Development Environment: Visual Studio Code or any text editor

Web Browser: Google Chrome, Mozilla Firefox or Microsoft Edge

Prototype storage: Browser Local Storage

Production requirements: Secure backend, database, identity/OTP provider, cryptographic key management and independent security auditing

4. Problem Description

Paper-based and manual election processes can require significant time and resources for voter verification, ballot distribution, vote collection and result compilation. VoteOnline demonstrates a centralized interface for administering an election and allowing eligible voters to authenticate and cast a vote remotely. The administrator creates elections and candidates, allocates election data to candidates, assigns users to wards, creates/imports voter lists, updates voter records and views results. Voters log in using their Voter ID, complete OTP verification, request/receive a voting OTP, select a candidate and submit their vote. A production system must protect ballot secrecy, prevent duplicate voting, provide verifiable auditability and comply with applicable election laws.

5. Modules

A. Admin Module

Create and configure an election and its candidates.

Allocate election data/details to the corresponding candidates.

Allocate registered voters to election wards.

Create and import the voter list.

View and update voter list records.

Get election results after the authorized counting process.

B. User / Voter Module

Login using a unique Voter ID.

Complete OTP login verification.

Request and receive a separate OTP/authorization for voting where required.

View candidate names and details for the assigned election/ward.

Select one candidate and submit the vote.

Receive a vote-submission confirmation without exposing the secret ballot choice.

6. Algorithm

Start the VoteOnline application.

Administrator authenticates and creates an election, candidates, wards and voter list.

Import or create voter records and allocate each eligible voter to the correct ward/election.

Voter enters the Voter ID on the login page.

System validates voter eligibility and initiates OTP verification.

Voter enters the OTP and the system verifies it.

Display the candidates available for the voter’s assigned election/ward.

Voter selects one candidate and confirms the submission.

System validates eligibility and duplicate-vote rules, records the ballot securely and issues a confirmation reference.

Administrator views election information and obtains results only through the authorized tallying process.

Stop.

7. Complete HTML, CSS and JavaScript Code

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>VoteOnline - E-Voting Portal</title>

<style>

*{box-sizing:border-box}body{margin:0;background:#f1f5f9;color:#172033;font:15px/1.5 Arial,sans-serif}

header{background:linear-gradient(110deg,#14213d,#1d4ed8);color:white;padding:22px max(5vw,20px);display:flex;justify-content:space-between;align-items:center;gap:14px;flex-wrap:wrap}

.brand{font-size:26px;font-weight:800}.brand span{color:#93c5fd}header small{color:#dbeafe}

main{max-width:1150px;margin:28px auto;padding:0 18px}.intro{display:flex;justify-content:space-between;align-items:center;gap:15px;flex-wrap:wrap}

h1{font-size:28px;margin:0 0 5px}h2{font-size:19px;color:#172554;margin:0 0 14px}.muted{color:#64748b}

.grid{display:grid;grid-template-columns:repeat(3,1fr);gap:14px;margin:20px 0}

.card,.panel,.stat{background:white;border:1px solid #dbe3ef;border-radius:14px;padding:18px;box-shadow:0 3px 12px #0f172a08}

.stat strong{display:block;font-size:27px;color:#172554}.stat span{color:#64748b;font-size:13px}

.layout{display:grid;grid-template-columns:1fr 1fr;gap:18px}

label{display:block;font-size:13px;font-weight:700;margin:11px 0 5px}

input,select{width:100%;padding:10px;border:1px solid #cbd5e1;border-radius:8px;font:inherit}

button{border:0;border-radius:8px;padding:11px 15px;background:#2563eb;color:white;font-weight:700;cursor:pointer;margin-top:12px}

button.secondary{background:#e2e8f0;color:#1e293b}button:disabled{opacity:.5;cursor:not-allowed}

.candidate{border:1px solid #dbe3ef;border-radius:11px;padding:13px;margin:10px 0;display:flex;gap:12px;align-items:center}
```

```
.avatar{width:44px;height:44px;border-radius:50%;background:#dbeafe;color:#1d4ed8;display:grid;place-items:center;font-weight:800;font-size:18px}

.candidate label{margin:0;flex:1}.candidate input{width:auto}

.notice{background:#eff6ff;border-left:4px solid #2563eb;padding:11px;border-radius:6px;color:#1e40af;font-size:12px;margin-top:12px}

table{width:100%;border-collapse:collapse;font-size:13px}td,th{padding:10px;text-align:left;border-bottom:1px solid #e2e8f0}th{color:#64748b}

.pill{background:#dcfce7;color:#166534;padding:4px 9px;border-radius:20px;font-size:11px;font-weight:bold}

@media(max-width:760px){.grid,.layout{grid-template-columns:1fr}}

</style></head>

<body>

<header><div><div class="brand">Vote<span>Online</span></div><small>Secure and transparent election portal</small></div><div>Voter portal · Demonstration</div></header>

<main>

<div class="intro"><div><h1>Welcome, registered voter</h1><div class="muted">Verify your voter ID, request an OTP and cast your vote.</div></div><span class="pill">Election status: Open</span></div>

<div class="grid">

<div class="stat"><strong>3</strong><span>Registered candidates</span></div>

<div class="stat"><strong>1,248</strong><span>Registered voters</span></div>

<div class="stat"><strong>Open</strong><span>Current election status</span></div>

</div>

<div class="layout">

<section class="panel"><h2>Voter verification</h2>

<label for="voterId">Voter ID</label><input id="voterId" placeholder="Enter your voter ID" autocomplete="off">

<button onclick="sendOtp()">Get OTP</button>

<label for="otp">One-time password (OTP)</label><input id="otp" inputmode="numeric" maxlength="6" placeholder="Enter 6-digit OTP">

<button class="secondary" onclick="verifyOtp()">Verify OTP</button>

<p id="verifyMsg" class="muted" aria-live="polite">Complete verification before voting.</p>
```

Demo OTP is displayed in a browser alert. A real system must deliver OTP through a trusted SMS/email provider and verify it on the server.

</section>

<section class="panel"><h2>Select your candidate</h2>

<form onsubmit="castVote(event)">

AK <label>Arun Kumar
Community Development Party</label><input type="radio" name="candidate" value="Arun Kumar" required>

MS <label>Meera Sharma
People's Progress Alliance</label><input type="radio" name="candidate" value="Meera Sharma">

RJ <label>Rahul Joshi
Public Welfare Front</label><input type="radio" name="candidate" value="Rahul Joshi">

<button id="voteBtn" type="submit" disabled>Submit Vote</button>

</form>

</section>

<section class="panel" style="margin-top:18px"><h2>Election administration preview</h2>

Administrator functions include creating elections and candidates, allocating election data to candidates, assigning voters to wards, creating/importing voter lists, viewing/updating voter records and retrieving election results.

<table><thead><tr><th>Election</th><th>Ward</th><th>Voter list</th><th>Status</th></tr></thead><tbody><tr><td>Municipal Election 2026</td><td>Ward 04</td><td>Imported · 1,248 voters</td><td>Open</td></tr></tbody></table>

This is an educational front-end simulation. It does not create a legally valid vote or provide election-grade security.

</section>

</main>

<script>

let demoOtp=null, verified=false, hasVoted=localStorage.getItem('vo_demo_voted')==='yes';

function sendOtp(){const id=document.getElementById('voterId').value.trim();if(!id){alert('Enter your voter ID first.');return}demoOtp=String(Math.floor(100000+Math.random()*900000));alert('Demo

OTP: '+demoOtp);document.getElementById('verifyMsg').textContent='OTP generated for demonstration. Enter it above.'}

function verifyOtp(){const entered=document.getElementById('otp').value.trim();if(demoOtp&&entered===demoOtp){verified=true;document.getElementById('verifyMsg').textContent='Voter ID and demo OTP verified.';document.getElementById('voteBtn').disabled=hasVoted}else{verified=false;document.getElementById('verifyMsg').textContent='OTP invalid. Request a new OTP and try again.'}}

function castVote(e){e.preventDefault();if(!verified){alert('Verify your voter ID with OTP first.');return}if(hasVoted){alert('A demo vote has already been recorded in this browser.');return}const selected=document.querySelector('input[name="candidate"]:checked');if(!selected){alert('Select a candidate.');return}hasVoted=true;localStorage.setItem('vo_demo_voted','yes');document.getElementById('voteBtn').disabled=true;document.getElementById('voteMsg').textContent='Demo vote submitted successfully. Confirmation reference: VO-DEMO-2026.';alert('Thank you. Your demo vote has been recorded.')}

if(hasVoted){document.getElementById('voteMsg').textContent='A demo vote was already recorded in this browser.';document.getElementById('voteBtn').disabled=true}

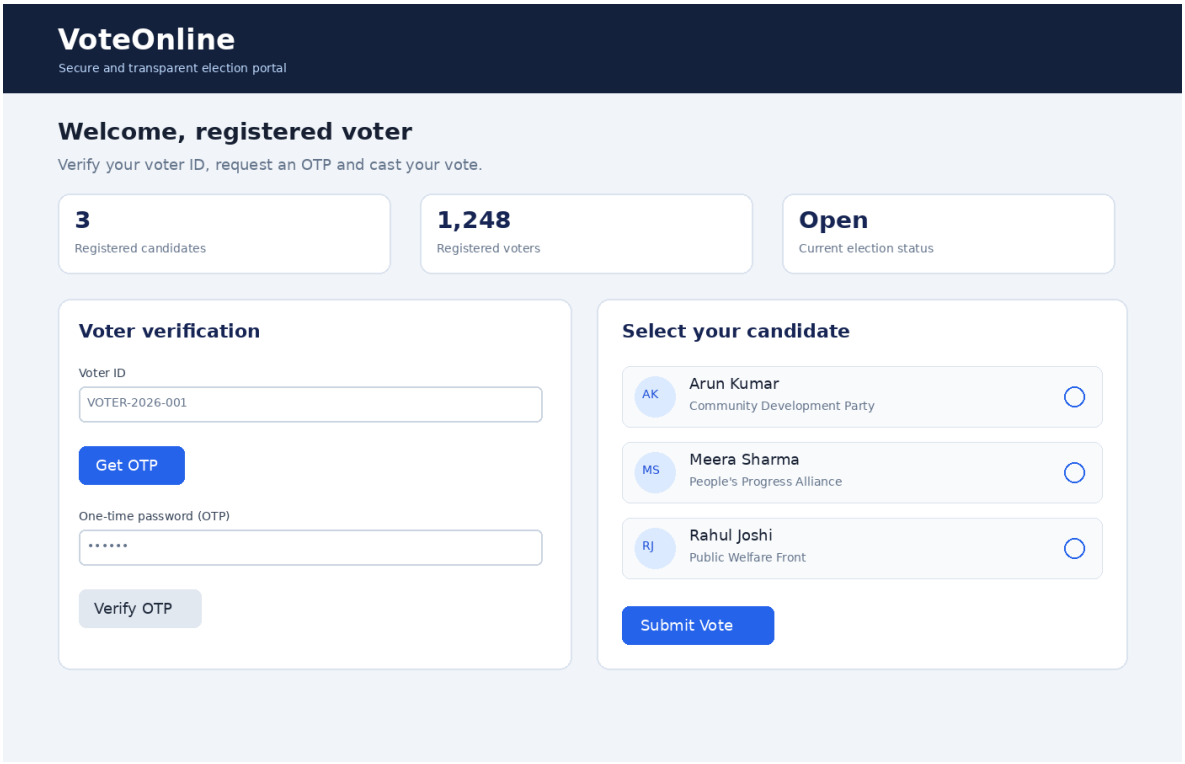
</script></body></html>

8. Sample Input

Field	Sample value
Election	Municipal Election 2026
Voter ID	VOTER-2026-001
OTP	Displayed demo OTP
Ward	Ward 04
Candidate selected	Meera Sharma
Confirmation	VO-DEMO-2026

9. Expected Output with Image

The expected screen displays election summary cards, voter ID and OTP verification fields, candidate options and a Submit Vote control. On successful demo verification, a voter can submit one simulated vote in the current browser. The screenshot is an illustrative output of the lab prototype.



10. Result

Thus, the VoteOnline e-voting interface prototype was designed using HTML, CSS and JavaScript. It demonstrates the basic voter authentication flow, simulated OTP verification, candidate selection and vote confirmation, together with the administrator workflow requirements. This prototype is not suitable for conducting a real election. A real deployment requires independently reviewed end-to-end security, server-side authentication and authorization, secure OTP delivery, encrypted and secret ballot storage, robust duplicate-vote prevention, tamper-evident audit logs, accessibility testing, disaster recovery and compliance with applicable electoral laws and official procedures.

Experiment-18

AGRIMART – ONLINE AGRICULTURAL MARKETPLACE

1. Title

AgriMart – Online Agricultural Products Shopping and Management System

2. Aim

To design and develop an online agricultural marketplace where customers can register, verify their login using OTP, search and purchase agricultural products, manage their cart and wallet, maintain shipping addresses and track orders. Farmers can manage product listings, images, orders, sales and shipping status.

3. Software Requirements

Operating System: Windows, Linux or macOS

Frontend: HTML5, CSS3 and JavaScript

Development Environment: Visual Studio Code or any text editor

Web Browser: Google Chrome, Mozilla Firefox or Microsoft Edge

Prototype storage: Browser session and Local Storage where applicable

Production requirements: Backend application, database, secure payment gateway, OTP service and image/file storage

4. Problem Description

Farmers and agricultural customers need convenient access to seeds, fertilizers, pesticides, equipment and other farming supplies. Traditional purchasing methods can make product discovery, price comparison, ordering and shipment tracking difficult. AgriMart provides a centralized e-commerce platform. Public visitors can register. Customers can log in with OTP verification, manage their account and wallet, view and search product details, add/edit/remove cart items, manage shipping addresses, place orders and check order/shipping status. Farmers can register/login, add products, upload product images, update or delete listings, manage incoming orders, update order and shipping status, view sales and log out.

5. Modules

A. Public Module

Public visitor registration to create a customer or farmer account.

B. Customer Module

Login and OTP verification.

View and update My Account information.

Manage wallet balance.

View detailed product information and search products.

Add products to cart and edit quantities or delete cart items.

Add or edit shipping addresses.

Place product orders.

View order status and shipping status.

C. Farmer Module

Farmer registration, login and logout.

OTP verification during login.

Add products and upload product images.

Update or delete product listings.

Manage customer orders and update order status.

Update shipment status.

View My Sales and order information.

6. Algorithm

Start the AgriMart website and display the home page and product categories.

Allow a public visitor to register as a customer or farmer.

Authenticate the user and verify the OTP.

For customers, display the product catalog and enable search and product-detail viewing.

Allow the customer to add products to the cart, change quantities or remove items.

Allow the customer to manage wallet balance and shipping address.

Validate cart, customer details and shipping address before placing an order.

Generate an order reference and display order and shipping status.

For farmers, allow product creation, image upload, update and deletion.

Allow farmers to view orders, update order/shipping status and view sales.

Stop.

7. Complete HTML, CSS and JavaScript Code

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>AgriMart - Agricultural Marketplace</title>

```
<style>
*{box-sizing:border-box}body{margin:0;background:#f4f7f1;color:#1d2b20;font:15px/1.5 Arial,sans-serif}
header{background:#174b2b;color:#fff;padding:20px max(5vw,20px);display:flex;justify-content:space-between;align-items:center;gap:15px;flex-wrap:wrap}
.brand{font-size:27px;font-weight:800}.brand span{color:#b7e4a8}header small{color:#d9f4d1}
main{max-width:1180px;margin:26px auto;padding:0 18px}.hero{background:linear-gradient(110deg,#e2f3df,#f8fbf4);padding:24px;border-radius:15px;display:flex;justify-content:space-between;align-items:center;gap:15px;flex-wrap:wrap}
h1{font-size:28px;margin:0 0 6px;color:#174b2b}h2{font-size:19px;margin:0 0 14px;color:#174b2b}.muted{color:#647568}
.toolbar{display:flex;gap:10px;flex-wrap:wrap;margin:18px 0}.toolbar input{flex:1;min-width:190px}
input,select,textarea{padding:10px;border:1px solid #cbd5c9;border-radius:8px;font:inherit;background:#fff}
button{border:0;border-radius:8px;padding:10px 14px;background:#287a3d;color:white;font-weight:700;cursor:pointer}
button.secondary{background:#e4eee3;color:#21452a}button.danger{background:#fee2e2;color:#991b1b}
.layout{display:grid;grid-template-columns:1fr 320px;gap:18px;align-items:start}
.products{display:grid;grid-template-columns:repeat(3,1fr);gap:14px}
.product{background:#fff;border:1px solid #dce7d9;border-radius:13px;overflow:hidden}
.photo{height:130px;background:linear-gradient(135deg,#e5f4df,#c9e5bd);display:grid;place-items:center;font-size:48px}
.product .body{padding:13px}.product h3{font-size:16px;margin:0 0 4px}.price{font-weight:800;color:#176534;font-size:18px}
.product small{color:#647568}.product button{width:100%;margin-top:9px}
.panel{background:#fff;border:1px solid #dce7d9;border-radius:13px;padding:17px;margin-bottom:15px}
label{display:block;font-size:13px;font-weight:700;margin:10px 0 5px}
.cart-row{display:flex;justify-content:space-between;gap:8px;padding:9px 0;border-bottom:1px solid #e5ece3;font-size:13px}
.total{font-size:19px;font-weight:800;color:#174b2b;margin:12px 0}
table{width:100%;border-collapse:collapse;font-size:13px}td,th{text-align:left;padding:9px;border-bottom:1px solid #e5ece3}th{color:#647568}
```

```
.badge{background:#e5f4df;color:#176534;padding:3px 8px;border-radius:20px;font-size:11px;font-weight:bold}

.note{font-size:12px;color:#526c56;background:#f0f7ed;padding:10px;border-left:4px solid #6ca56b;border-radius:5px;margin-top:10px}

@media(max-width:850px){.layout{grid-template-columns:1fr}.products{grid-template-columns:repeat(2,1fr)}}

@media(max-width:520px){.products{grid-template-columns:1fr}}

</style></head>

<body>

<header><div><div class="brand">Agri<span>Mart</span></div><small>Your trusted agricultural marketplace</small></div><div>Farmer & customer portal · Demo</div></header>

<main>

<section class="hero"><div><h1>Grow better with AgriMart 🌱</h1><div class="muted">Seeds, fertilizers, crop protection and farming tools in one place.</div></div><div><span class="badge">Demo store</span></div></section>

<div class="toolbar"><input id="search" placeholder="Search seeds, fertilizer, tools..." oninput="renderProducts()"><select id="category" onchange="renderProducts()"><option value="All">All categories</option><option>Seeds</option><option>Fertilizers</option><option>Pesticides</option><option>Equipment</option></select></div>

<div class="layout"><section><div class="products" id="products"></div>

<section class="panel" style="margin-top:18px"><h2>Farmer product management</h2><form onsubmit="addProduct(event)"><div class="row"><label>Product name<input id="newName" required placeholder="e.g. Tomato seeds"></label><label>Category<select id="newCategory"><option>Seeds</option><option>Fertilizers</option><option>Pesticides</option><option>Equipment</option></select></label></div><div class="row"><label>Price (₹)<input id="newPrice" type="number" min="1" required></label><label>Image file<input id="newImage" type="file" accept="image/*"></label></div><button type="submit">Add product</button></form><div class="note">Farmer demo supports adding products and deleting demo-added items. Full image uploads, editing, sales, shipping and order management need a backend.</div></section>

</section>

<aside>

<div class="panel"><h2>My Cart</h2><div id="cartItems" class="muted">Your cart is empty.</div><div class="total" id="total">Total: ₹0</div><button
```

onclick="checkout()">Order Product</button><button class="secondary" onclick="clearCart()">Clear cart</button>

<h2>Account & wallet</h2><label for="customer">Customer name</label><input id="customer" placeholder="Enter your name"><label for="wallet">Wallet balance (₹)</label><input id="wallet" type="number" value="1000" min="0"><button class="secondary" onclick="alert('Demo wallet balance: ₹'+document.getElementById('wallet').value)">View wallet</button> Wallet and checkout are simulations; no real payment is processed.

<h2>Shipping address</h2><textarea id="address" placeholder="House, village, district, PIN"></textarea><button class="secondary" onclick="saveAddress()">Save address</button>

</aside>

<section class="panel" style="margin-top:18px"><h2>My Orders & shipping status</h2> <table><thead><tr><th>Order ID</th><th>Items</th><th>Amount</th><th>Order status</th><th>Shipping</th></tr></thead><tbody id="orders"><tr><td>AG-1001</td><td>Hybrid maize seeds</td><td>₹650</td><td>Confirmed</td><td>Dispatched</td></tr></tbody></table> </section>

</main>

<script>

let catalog=[

{id:1,name:'Hybrid Maize Seeds',category:'Seeds',price:650,emoji:'🌽'},

{id:2,name:'Organic Vermicompost',category:'Fertilizers',price:420,emoji:'🌱'},

{id:3,name:'Neem-Based Crop Spray',category:'Pesticides',price:310,emoji:'🍃'},

{id:4,name:'Hand Sprayer 5L',category:'Equipment',price:890,emoji:'🧴'},

{id:5,name:'Tomato Seed Pack',category:'Seeds',price:180,emoji:'🍅'},

{id:6,name:'Drip Irrigation Kit',category:'Equipment',price:1450,emoji:'💧'}];

let cart=[],orders=[],nextId=7;

function renderProducts(){let q=document.getElementById('search').value.toLowerCase(),cat=document.getElementById('category').value;let list=catalog.filter(p=>(cat==='All'||p.category===cat)&&(p.name.toLowerCase().includes(q)||p.category.toLowerCase().includes(q)));document.getElementById('products').innerHTML=list.map(p=>`<article class="product"> ${p.emoji}</div><div class="body"><h3>${p.name}</h3><small>${p.category}</small><div

```
class="price">₹${p.price}</div><button onclick="addCart(${p.id})">Add to cart</button>${p.id>=7?`<button class="danger" onclick="deleteProduct(${p.id})">Delete product</button>`:''}</div></article>`).join('')||'<p class="muted">No products found.</p>'}

function addCart(id){let p=catalog.find(x=>x.id===id);let item=cart.find(x=>x.id===id);if(item)item.qty++;else cart.push({...p,qty:1});renderCart()}

function renderCart(){let el=document.getElementById('cartItems');el.innerHTML=cart.length?cart.map(i=>`<div class="cart-row"><span>${i.name}<br>₹${i.price} × <button class="secondary" onclick="changeQty(${i.id},-1)">−</button> ${i.qty} <button class="secondary" onclick="changeQty(${i.id},1)">+</button></span><button class="danger" onclick="removeCart(${i.id})">Remove</button></div>`).join(''):'Your cart is empty.';document.getElementById('total').textContent='Total: ₹'+cart.reduce((s,i)=>s+i.price*i.qty,0)}

function changeQty(id,n){let x=cart.find(i=>i.id===id);if(x){x.qty+=n;if(x.qty<1)removeCart(id);else renderCart()}}

function removeCart(id){cart=cart.filter(i=>i.id!==id);renderCart()}

function clearCart(){cart=[];renderCart()}

function checkout(){if(!cart.length){alert('Add a product to your cart first.');return}let name=document.getElementById('customer').value.trim(),addr=document.getElementById('address').value.trim();if(!name||!addr){alert('Enter customer name and shipping address.');return}let amount=cart.reduce((s,i)=>s+i.price*i.qty,0);if(Number(document.getElementById('wallet').value)<amount){alert('Insufficient demo wallet balance.');return}let id='AG-'+(1002+orders.length);orders.push({id,items:cart.map(i=>i.name+' × '+i.qty).join(', '),amount});let tr=document.createElement('tr');tr.innerHTML=`<td>${id}</td><td>${orders[orders.length-1].items}</td><td>₹${amount}</td><td><span class="badge">Placed</span></td><td>Processing</td>`;document.getElementById('orders').prepend(tr);document.getElementById('wallet').value=Number(document.getElementById('wallet').value)-amount;clearCart();alert('Demo order placed: '+id)}

function addProduct(e){e.preventDefault();catalog.push({id:nextId++,name:document.getElementById('newName').value,category:document.getElementById('newCategory').value,price:Number(document.getElementById('newPrice').value),emoji:'🌾'});e.target.reset();renderProducts();alert('Product added in this demo.')}

function deleteProduct(id){catalog=catalog.filter(p=>p.id!==id);renderProducts()}

function saveAddress(){let a=document.getElementById('address').value.trim();document.getElementById('addressMsg').textContent=a?'Shipping address saved for this session.':'Please enter an address.'}

renderProducts();renderCart();
```

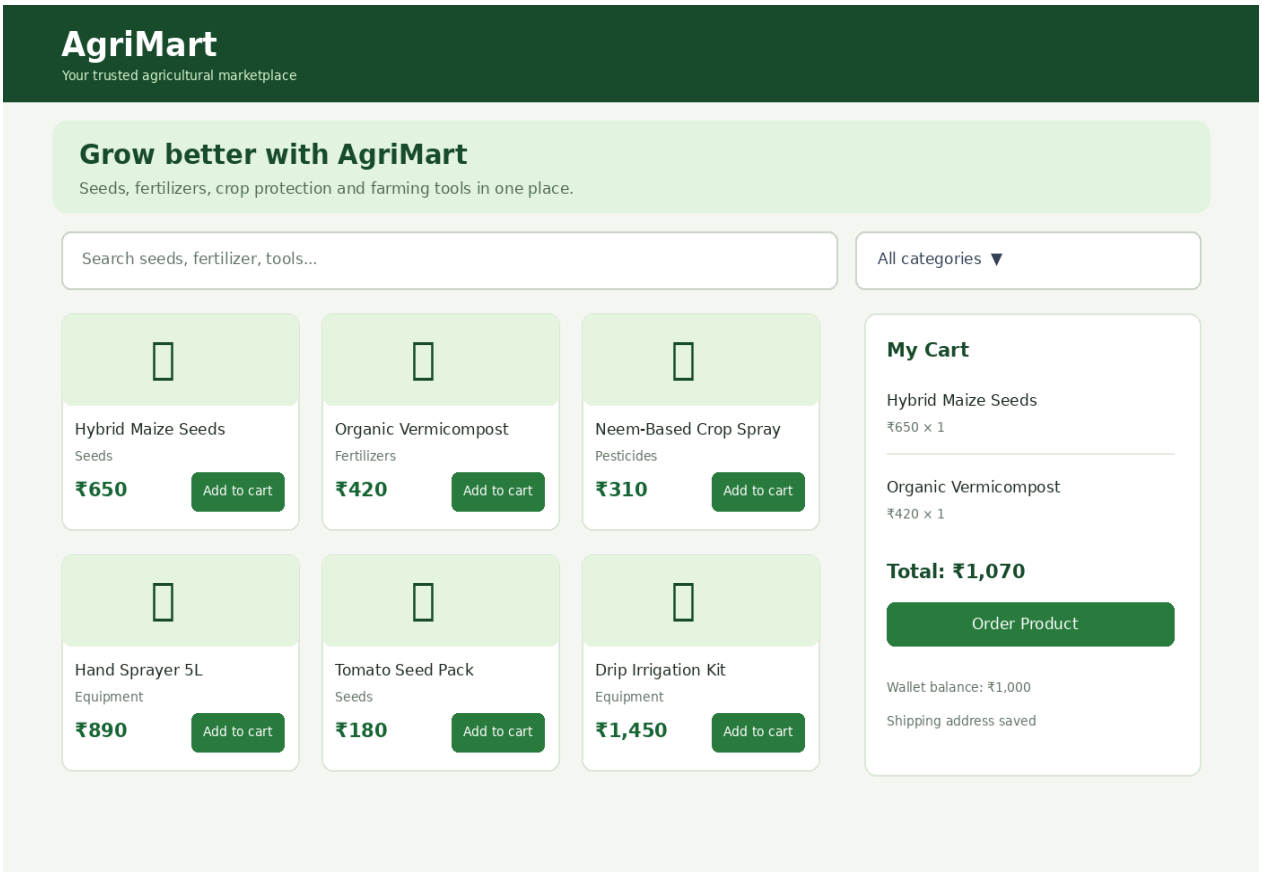
</script></body></html>

8. Sample Input

Field	Sample value
Customer name	Ravi Kumar
Search keyword	Seeds
Category	Seeds
Product	Hybrid Maize Seeds
Quantity	1
Wallet balance	₹1,000
Shipping address	Village Road, Ward 2, District, PIN
Order reference	AG-1002

9. Expected Output with Image

The expected output shows the AgriMart product catalog with category/search controls, agricultural product cards and prices, a cart with quantity controls and total, wallet balance and shipping address area. The farmer product-management form and order-status table are available further down the page.



10. Result

Thus, the AgriMart online agricultural marketplace prototype was designed using HTML, CSS and JavaScript. It demonstrates product browsing, search/filtering, cart management, wallet and shipping-address interfaces, demo order placement and basic farmer product management. The prototype is intended for Internet Programming lab demonstration. Production deployment requires secure server-side authentication and OTP verification, persistent database storage, protected image uploads, payment gateway integration, inventory management, reliable order/shipping services and appropriate security controls.